Introduction

Hello coders! Let's start our quest with **regular expressions (RegEx)**. In Python, the module `re` provides full support for **Perl**-like regular expressions in Python. We need to remember that there are many characters in Python, which would have special meaning when they are used in regular expression. To avoid bugs while dealing with regular expressions, we use **raw** strings as `r'expression'`.

The `re` module in Python provides multiple methods to perform queries on an input string. Here are the most commonly used methods:

```
re.match()
re.search()
re.split()
re.sub()
re.findall()
re.compile()
```

We will look at these function and related flags with examples in the next section.

In case you do not know what's this all about, it is a special sequence of characters or **pattern** that helps us to **match** or **find** other strings or sets of strings, using a specialized syntax held in a pattern. Regular expressions are widely used in the UNIX world. This course assumes that you have basic understanding of the REGEX patterns, if you don't, not to worry, we'll introduce with some REGEX basics in the next lesson!

Python Regular Expression Patterns List

Learn about the Python REGEX symbols

WE'LL COVER THE FOLLOWING

- Groups and Lookarounds

The following table lists the regular expression syntax that is available in Python. Note that any Regex can be concatenated to form new regular expressions; if `X` and `Y` are both regular expressions, then `XY` is also a regular expression.

Pattern	Description
`.`	Matches any single character except newline. Using `m` option allows it to match newline as well.
`^`	Matches the start of the string, and in `re.MULTILINE` (see the next lesson on how to change to multiline) mode also matches immediately after each newline.
`$`	Matches end of line. In `re.MULTILINE` mode also matches before a newline.
`[.]`	Matches any single character in brackets.

`[^.]`	Matches any single character not in brackets.
`*`	Matches 0 or more occurrences of preceding expression.
`+`	Matches 1 or more occurrence of preceding expression.
`?`	Matches 0 or 1 occurrence of preceding expression.
`{ n}`	Matches exactly `n` number of occurrences of preceding expression.
`{ n,}`	Matches `n` or more occurrences of preceding expression.
`{ n, m}`	Matches at least `n` and at most `m` occurrences of preceding expression. For example, `x{3,5}` will match from `3` to `5` `'x'` characters.
`x| y`	Matches either `x` or `y`.
`\d`	Matches digits. Equivalent to `[0-9]`.
`\D`	Matches nondigits.
`\w`	Matches word characters.
`\W`	Matches nonword characters.

`\z`	Matches end of string.
`\G`	Matches point where last match finished.
`\b`	Matches the empty string, but only at the beginning or end of a word. Boundary between word and non-word and `/B` is opposite of `/b`. Example `r"\btwo\b"` for searching two from `'one two three'`.
`\B`	Matches nonword boundaries.
`\n, \t`	Matches newlines, carriage returns, tabs, etc.
`\s`	Matches whitespace.
`\S`	Matches nonwhitespace.
`\A`	Matches beginning of string.
`\Z`	Matches end of string. If a newline exists, it matches just before newline.

Groups and Lookarounds

More details later:

Pattern	Description
`(re)`	Groups regular expressions and remembers matched text.

`(?: re)`	Groups regular expressions without remembering matched text. For example, the expression `(?:x{6})*` matches any multiple of six ‘`x`’ characters.
`(?#...)`	Comment.
`(?= ...)`	Matches if `...` matches next, but doesn’t consume any of the string. This is called a **lookahead assertion**. For example, `Scientific(?=Python)` will match `Scientific` only if it’s followed by `Python`.
`(?!...)`	Matches if `...` doesn’t match next. This is a negative lookahead assertion.
`(?<=...)`	Matches if the current position in the string is preceded by a match for `...` that ends at the current position.

Quiz # 1 - REGEX Patterns

Challenge yourself!

Regex pattern quizzes

1 By default, a dot (`.`) matches

2 Regex pattern `a+` matches

3

Regex [0-9]+ matches one or more occurrence of any digit

4

[^...] Matches any single character that is not in the class.

5

^abc matches - multi answer

6

The vertical bar separates two or more alternatives. A match occurs if any of the alternatives is satisfied. For example, `learn |scientific` matches

7

By default, regular expressions are case-sensitive.

8

When you put a plus sign (`+`) after something in a regular expression,

9

The star (`*`) has a similar meaning but also allows the pattern to match zero times.

10

Putting `{4}` after an element, `\d{4}`

Check Answers

Python regex `match` function

Python Regex match() function explained with examples.

WE'LL COVER THE FOLLOWING

- The Match Function
- Match Flags
- Return values
- Example 1
- Example 2
- Example 3
  - Example 4
  - Example 5

The Match Function

The `match` function attempts to match a `re` pattern to string with optional flags.

Here is the syntax for this function –

```
re.match(pattern, string, flags=0)
```

Where, `pattern` is the regular expression to be matched, `string` is the string to be searched to match the pattern at the beginning of string and `flags`, which you can specify different flags using bitwise OR (`|`).

Match Flags

Modifier	Description

`re.I`	Performs case-insensitive matching.
`re.L`	Interprets words according to the current locale. This interpretation affects the alphabetic group (`\w` and `\W`), as well as word boundary behavior (`\b` and `\B`).
`re.M`	Makes `$` match the end of a line and makes ^ match the start of any line.
`re.S`	Makes a period (dot) match any character, including a newline.
`re.U`	Interprets letters according to the Unicode character set. This flag affects the behavior of `\w`, `\W`, `\b`, `\B`.
`re.X`	It ignores whitespace (except inside a set `[]` or when escaped by a `backslash` and treats unescaped `#` as a comment marker.

Return values

- The `re.match` function returns a `match object` on **success** and `None` upon failure. -
- Use `group(n)` or `groups()` function of match object to get matched expression, e.g., `group(n=0)` returns entire match (or specific subgroup `n`)
- The function `groups()` returns all matching subgroups in a tuple (empty if there weren't any).

Example 1

Let's find the words before and after the word `to`:

```
#!/usr/bin/python
import re

line = "Learn to Analyze Data with Scientific Python";

m = re.match( r'(.*) to (.*?) .*', line, re.M|re.I)

if m:
   print "m.group() : ", m.group()
   print "m.group(1) : ", m.group(1)
   print "m.group(2) : ", m.group(2)
else:
   print "No match!!"
```

The first group `(.*)` identified the string: Learn and the next group `(*.?)` identified the string: Analyze.

Example 2

`groups([default])` returns a tuple containing all the subgroups of the match, from 1 up to however many groups are in the pattern.

```
#!/usr/bin/python
import re

line = "Learn Data, Python";

m = re.match( r'(\w+) (\w+)', line, re.M|re.I)

if m:
   print "m.group() : ", m.groups()
   print "m.group (1,2)", m.group(1, 2)
else:
   print "No match!!"
```

Example 3

`groupdict([default])` returns a dictionary containing all the named subgroups of the match, keyed by the subgroup name.

```
#!/usr/bin/python
```

```
import re

number = "124.13";

m = re.match( r'(?P<Expotent>\d+)\.(?P<Fraction>\d+)', number)

if m:
   print "m.groupdict() : ", m.groupdict()
else:
   print "No match!!"
```

Example 4

Start, end. How can we match the start or end of a string? We can use the “A” and “Z” metacharacters. We precede them with a backslash. We match strings that start with a certain letter, and those that end with another.

```
import re

values = ["Learn", "Live", "Python"];

for value in values:
    # Match the start of a string.
    result = re.match("\AL.+", value)
    if result:
        print("START MATCH [L]:", value)

    # Match the end of a string.
    result2 = re.match(".+n\Z", value)
    if result2:
        print("END MATCH [n]:", value)
```

Example 5

`start([group])` and `end([group])` return the indices of the start and end of the substring matched by `group`. See the next lesson for an example.

Python Regex Match ( ) function - Coding exercise

Test your Python Regex match() and group() function skills.

WE'LL COVER THE FOLLOWING

- The Match Function Exercise
- Do you need a hint?

The Match Function Exercise

Let's assume that we have a string variable called `line`, where `line = "Learn to Analyze Data with Scientific Python"` passed to a function called `regex_processor()` in the following code.

Can you find the word `"Analyze"` after the word `"to"`?

```
import re

def regex_processor(line):
  # Passed from the arg, line = "Learn to Analyze Data with Scientific Python"
  m = re.match( r'(.*) to (.*?) .*', line, re.M|re.I)

  # Replace the '0' with correct number (1 or 2?)
  if m:
     print  (m.group(0))
  else:
     print ("No match!!")
```

Regex processor function

Your task is to replace the `'?'` on the **line 9** with a correct group number (`1` or `2`) for the word `Analyze`.

Do you need a hint?

Click on the button below:

Click on the button below:

Show Hint

Python regex `search` function

Python Regex search() function explained with examples.

WE'LL COVER THE FOLLOWING

- The Search Function
- Match Flags
- Return values
- Example 1
- Example 2
- Example 3
  - Example 4
- Why we need to use `search` instead of the `match`?

The Search Function

The `search` function searches for first occurance of a `re` pattern to string with optional flags.

Here is the syntax for this function –

```
re.search(pattern, string, flags=0)
```

Where, `pattern` is the regular expression to be matched, `string` is the string to be searched to match the pattern at the beginning of string and `flags`, which you can specify different flags using bitwise OR (`|`).

Match Flags

Modifier	Description

`re.I`	Performs case-insensitive matching.
`re.L`	Interprets words according to the current locale. This interpretation affects the alphabetic group (`\w` and `\W`), as well as word boundary behavior (`\b` and `\B`).
`re.M`	Makes `$` match the end of a line and makes ^ match the start of any line.
`re.S`	Makes a period (dot) match any character, including a newline.
`re.U`	Interprets letters according to the Unicode character set. This flag affects the behavior of `\w`, `\W`, `\b`, `\B`.
`re.X`	It ignores whitespace (except inside a set `[]` or when escaped by a `backslash` and treats unescaped `#` as a comment marker.

Return values

- The `re.search` function returns a match `object` on **success** and `None` upon failure. -
- Use `group(n)` or `groups()` function of match object to get matched expression, e.g., `group(n=0)` returns entire match (or specific subgroup `n`)
- The function `groups()` returns all matching subgroups in a tuple (empty if there weren't any).

Example 1

Let's find the words before and after the word `to`:

```
#!/usr/bin/python
import re

line = "Learn to Analyze Data with Scientific Python";

m = re.search( r'(.*) to (.*?) .*', line, re.M|re.I)

if m:
   print "m.group() : ", m.group()
   print "m.group(1) : ", m.group(1)
   print "m.group(2) : ", m.group(2)
else:
   print "No match!!"
```

The first group `(.*)` identified the string: Learn and the next group `(*.?)` identified the string: Analyze.

Example 2

`groups([default])` returns a tuple containing all the subgroups of the match, from 1 up to however many groups are in the pattern.

```
#!/usr/bin/python
import re

line = "Learn Data, Python";

m = re.search( r'(\w+) (\w+)', line, re.M|re.I)

if m:
   print "m.group() : ", m.groups()
   print "m.group (1,2)", m.group(1, 2)
else:
   print "No match!!"
```

Example 3

`groupdict([default])` returns a dictionary containing all the named subgroups of the match, keyed by the subgroup name.

```
#!/usr/bin/python
```

```
import re

number = "124.13";

m = re.search( r'(?P<Expotent>\d+)\.(?P<Fraction>\d+)', number)

if m:
   print "m.groupdict() : ", m.groupdict()
else:
   print "No match!!"
```

Example 4

`start([group])` and `end([group])` return the indices of the start and end of the substring matched by `group`. We need to use `search` instead of `match` for this example:

```
#!/usr/bin/python
import re

email = "hello@leremove_thisarntoanalayzedata.com ";

# m = re.match ("remove_this", email) // This will not work!
m = re.search("remove_this", email)

if m:
   print "email address : ", email[:m.start()] + email[m.end():]
else:
   print "No match!!"
```

Why we need to use `search` instead of the `match`?

Can you guess from the example above?

Show Hint

See in the next lesson!

Python Regex search ( ) function - Coding exercise

Test your Python Regex search() and group() function skills.

WE'LL COVER THE FOLLOWING

- The Search Function Exercise

The Search Function Exercise

Let's assume that we have a string variable called `email`, where `email = "hello@scitake_outentificprogramming.io"` passed to a function called `regex_processor()` in the following code.

Can you remove the word `"take_out"` and print the email address `"hello@scientificprogramming.io"`?

```
#!/usr/bin/python
import re

def regex_processor(email):

  #email = "hello@scitake_outentificprogramming.io ";
  # replace 'match ()' with the appropriate function
  m = re.match("take_out", email)

  # replace '0' with the 'm.start()' and 'm.end()' at the appropriate indices?
  if m:
    print (email[:0] + email[0:])
  else:
    print ("No match!!")
```

Regex processor function

Your task is to replace the `'?'` on the **line 8** with the correct function name (`match` or `search` ?) and replace he `?` marks on the **line 12** with correct match object function calls (hint: `m.end()` and `m.start()`).

Show Hint

Python regex `match` vs. `search` functions

Difference between Python regex match() and search() function!

WE'LL COVER THE FOLLOWING

- Python Matching Versus Searching
- Example 1
- Example 2:

Python Matching Versus Searching

We have learned so far that Python offers two different **primitive** operations:

- `match`
- `search`

So, how they are different to each other?

Note that `match` checks for a match only at the beginning of a string, while `search` checks for a match anywhere in the string!

Example 1

Let's try to find the word `Python`:

```
#!/usr/bin/python
import re

line = "Learn to Analyze Data with Scientific Python";

m = re.search( r'(python)', line, re.M|re.I)

if m:
   print "m.group() : ", m.group()
else:
   print "No match by obj.search!!"

m = re.match( r'(python)', line, re.M|re.I )
```

```
if m:
  print "m.group() : ", m.group()


else:
  print "No match by obj.match"
```

You will be able to see that, `match` function won't find the word "Python", but search can! Also note the use of the `re.I` (case insensitive) option.

Example 2:

```
#!/usr/bin/python
import re

email = "hello@leremove_thisarntoanalayzedata.com ";

# m = re.match ("remove_this", email) // This will not work!
m = re.search("remove_this", email)

if m:
   print "email address : ", email[:m.start()] + email[m.end():]
else:
   print "No match!!"
```

Python regex `group` functions

Python Regex group() function explained with examples: named groups and groupdict.

WE'LL COVER THE FOLLOWING

- Group dictionary `Groupdict`

A regular expression can have **named** groups. This makes it easier to retrieve those groups after calling `match()`. But it makes the pattern more complex.

Following example shows a named group (`first` and `last`).

```
#!/usr/bin/python
import re

# A string.
name = "Learn Scientific"

# Match with named groups.
m = re.match("(?P<first>\w+)\W+(?P<last>\w+)", name)

# Print groups using names as id.
if m:
    print(m.group("first"))
    print(m.group("last"))
```

We can get the first name with the string **"first"** and the `group()` method. We use **"last"** for the last name.

Group dictionary `Groupdict`

A regular expression with named groups can fill a dictionary. This is done with the `groupdict()` method. In the dictionary, each group name is a `key` and Each value is the data matched by the regular expression. So we receive a key-

value store based on groups.

```
import re

name = "Scientific Python"

# Match names.
m = re.match("(?P<first>\w+)\W+(?P<last>\w+)", name)

if m:
    # Get dict.
    d = m.groupdict()

    # Loop over dictionary with for-loop.
    for t in d:
        print("  key:", t)
        print("value:", d[t])
```

Python regex `sub` function for search and replace

Python regex substitute function.

WE'LL COVER THE FOLLOWING

- Python search and replace
- Syntax
- Example 1
- Example 2:

Python search and replace

The `sub()` function replaces every occurrence of a pattern with a string or the result of a function.

Syntax

```
re.sub(pattern, repl, string, maximum=0)
```

This method replaces all occurrences of the `re` `pattern` in `string` with `repl`, substituting all occurrences unless a `maximum` value is provided. Finally, returns the modified string.

Example 1

Format the phone number **+61-927 479-548**, remove everything except the digits:

```
#!/usr/bin/python
import re

phone = "Please call the phone # +61-927 479-548"

# Remove anything other than digits
num = re.sub(r'\D', "", phone)
print "The raw phone numbe is : ", num
```

Example 2:

Let's use the `sub()` function to "munge" a text, i.e., **randomize** the order of all the characters in each word of a sentence except for the first and last characters:

```
#!/usr/bin/python
import re
import random

def repl(m):
        inner_word = list(m.group(2))
        random.shuffle(inner_word)
        return m.group(1) + "".join(inner_word) + m.group(3)

line = "Learn Scientific Python with Regex";
m = re.sub(r"(\w)(\w+)(\w)", repl, line);

if m:
   print "munged : ", m;
else:
   print "No match!!";
```

Python regex `split` function

Python regex split() function explained with examples.

WE'LL COVER THE FOLLOWING

- Python string splitter
- Syntax
- Example 1
- Example 2

Python string splitter

The `split()` funtion, splits a string by the occurrences of a `pattern`.

Syntax

```
re.split(pattern, string, maxsplit=0, flags=0)
```

If `maxsplit` is nonzero, at most `maxsplit` splits occur, and the remainder of the string is returned as the final element of the list.

Example 1

Break the string: '**Learn, Scientific, Python**', into three elements: `Learn`, `Scientific`, `Python`:

```
#!/usr/bin/python
import re

line = "Learn, Scientific, Python"

m = re.split('\W+', line)

if m:
  print m
else:
  print "No match!"
```

Example 2

Now let's make a second example, where a splitter can any alphabet from `[a-z]`

```
#!/usr/bin/python
import re

line = "+61Lean7489Scientific324234"

m = re.split('[A-Za-z]+', line, re.I)

if m:
  print m
else:
  print "No match!"
```

Python regex `findall` function

Python regex findall() function explained with examples.

WE'LL COVER THE FOLLOWING

- Python string findall
- Syntax
- Example 1
- Example 2: `findall` and Groups

Python string findall

`findall()` is a powerful function in the `re` module. It finds *all* the matches and returns them as a list of strings, with each string representing one match.

Syntax

```
re.findall(pattern, string, flags=0)
```

The string is scanned **left-to-right**, and matches are returned in the order found. If **one or more** groups are present in the pattern, return a `list of groups`. Empty matches are included in the result unless they touch the beginning of another match.

Example 1

Find all and return the email addresses:

```
#!/usr/bin/python
import re

line = 'your alpha@scientificprograming.io, blah beta@scientificprogramming.io blah user'

emails = re.findall(r'[\w\.-]+@[\w\.-]+', line)

if emails:
```

```
  print emails
else:
  print "No match!"
```

Example 2: `findall` and Groups

Now let's make a second example. Groups `()` can be combined with `findall()`. If the pattern includes 2 or more parenthesis groups, then instead of returning a list of strings, `findall()` returns a list of *tuples*. Each tuple represents one match of the pattern, and inside the tuple is the `group(1)`, `group(2)`, etc.

The following example, will find, `'alpha'`, `'scientificprograming.io'`, `'beta'`, and `'scientificprogramming.me'`.

```
#!/usr/bin/python
import re

line = 'your alpha@scientificprograming.io, blah beta@scientificprogramming.me blah user'

tuples = re.findall(r'([\w\.-]+)@([\w\.-]+)', line)

if tuples:
  print tuples
else:
  print "No match!"
```

Once you have the list of tuples, you can loop over it to do some computation for each tuple.

Python regex `compile` function

Python regex compile() function explained with examples. Includes learning tasks.

WE'LL COVER THE FOLLOWING

- Python regex compile
- Syntax
- Example
- Learning Tasks

Python regex compile

The `compile` function compiles a regular expression pattern into a regular expression `object`, which can be used for matching using its `match()`, `search()`, etc. methods.

Syntax

```
re.compile(pattern, flags=0)
```

The expression's behaviour can be modified by specifying a flags value (discussed earlier). Values can be any of the following variables, combined using bitwise OR (the `|` operator).

For example:

```
m = re.match(pattern, string)
```

is equivalent to:

```
p = re.compile(pattern)
m = p.match(string)
```

Note that the programs that use only **a few regular expressions at a time**

Note that the programs that use only **a few regular expressions at a time** don't need to compile regular expressions (recent patterns are cached automatically due to `re._MAXCACHE` setting).

Example

Consider that you have an html file index.html like below:

```
<html>
<header>SP:</header>
<body>
<h1>Learn</h1>
<p>Scientific Programming</p>
</body>
</html>
```

You want to read this file and output:

```
SP:Learn Scientific Programming
```

The following code can do this:

main.py

index.html

```
import re
import os
def main():
    f = open('index.html')
    pattern = re.compile(r'(?P<start><.+?>)(?P<content>.*?)(</.+?>)')
    output_text = []
    for text in f:
        match = pattern.match(text)
        if match is not None:
            output_text.append(match.group('content'))

    fixed_content = ' '.join(output_text)

    print fixed_content

    f.close()

if __name__ == '__main__':
    main()
```

Learning Tasks

- How the regex pattern `(?P<start><.+?>)(?P<content>.*?)(</.+?>)` matches the string `<h1>Learn</h1>`.
- Learn the use of the Python `.append` and `.join` functions. Hints: `join()` returns a string in which the string elements of sequence have been joined by `str` separator and `append()` Add an item to the end of the list; equivalent to `a[len(a):] = [x]`.

Python regex `finditer` function

Python regex finditer() function explained with examples.

WE'LL COVER THE FOLLOWING

- Python string finditer
- Syntax
- Example 1

Python string finditer

`finditer()` is a powerful function in the `re` module. It returns an iterator yielding `MatchObject` instances over all non-overlapping matches for the RE pattern in string.

Syntax

```
re.finditer(pattern, string, flags=0)
```

Here the string is scanned **left-to-right**, and matches are returned in the order found. Empty matches are included in the result unless they touch the beginning of another match.

Example 1

Here is a simple example which demonstrates the use of finditer. It reads in a page of html text, finds all the occurrences of the word "**the**" and prints "the" and the following word. It also prints the character position of each match using the MatchObject's `start()` method.

```
import re
import urllib2

html = urllib2.urlopen('https://docs.python.org/2/library/re.html').read()
pattern = r'\b(the\s+\w+)\s+'
```

```
regex = re.compile(pattern, re.IGNORECASE)
for match in regex.finditer(html):
    print "%s: %s" % (match.start(), match.group(1))
```

Once you have the list of tuples, you can loop over it to do some computation for each tuple.

Expected output:

```
output

3261: The Python
4210: the backslash
4451: the same
4474: the same
4651: the pattern
4679: the regular
4930: The solution
5937: The functions
6301: the standard
and so on...
```

Quiz # 2 Python REGEX Functions

Challenge yourself!

Python REGEX (RE) functions

1 When you deal with the HTML and XML you may need:

2 Split the string into a list, splitting it wherever the RE matches

3

We use `RE` **compile**

4

What is `(?=...)` - Multiple correct

5

Groups are marked by the `'(', ')'` metacharacters.

Check Answers

Basics

Look around basics

Lookarounds often cause confusion to new regex learners. There are four lookarounds:

```
(?<= … ) and (?= … ),

(?<! … ) and (?! … )
```

Collectively, lookbehinds and lookaheads are known as lookarounds. Let's see the following table of examples:

Lookaround	Name	What it Does
`(?=learn)`	Lookahead	Asserts that what immediately follows the current position in the string is `learn`
`(?<=learn)`	Lookbehind	Asserts that what immediately precedes the current position in the string is `learn`
`(?!learn)`	Negative Lookahead	Asserts that what immediately follows the current position in the string is not `learn`
`(?<!learn)`	Negative Lookbehind	Asserts that what immediately precedes

		the current position in the string is not `learn`

Python Lookahead

Python look ahead explained.

WE'LL COVER THE FOLLOWING

- Positive Lookahead
- Example
- Neagative Lookahead
- Example

Positive Lookahead

Python positive lookahead matches at a position where the pattern inside the lookahead can be matched. Matches only the position. It does not consume any characters or expand the match.

Example

Consider the following string:

`begin:learner1:scientific:learner2:scientific:learner3:end`

Positive lookahead assertion can help us to find all words followed by the word `scientific`.

```
import re

string = "begin:learner1:scientific:learner2:scientific:learner3:end"
print re.findall(r"(\w+)(?=:scientific)", string)
```

Note the output `learner1` and `learner2`, but not `learner3`, which is followed by the word `:end`.

Neagative Lookahead

Similar to positive lookahead, except that negative lookahead only succeeds if the regex inside the lookahead fails to match.

Example

Let's now proceed to an example, where we find the word (`learner3`) followed by `end`.

```
import re

string = "begin:learner1:scientific:learner2:scientific:learner3:end"
print re.findall(r"(learner\d+)(?!:scientific)", string)
```

This matched all the words, not followed by the word `scientific`!

Python Lookbehind

Python Lookbehind explained.

WE'LL COVER THE FOLLOWING

- Positive Lookbehind
- Example
- Neagative Lookbehind
- Example

Positive Lookbehind

`(?<=regex)` Matches at a position if the pattern inside the lookbehind can be matched ending at that position.

Example

Consider the following string:

`begin:learner1:scientific:learner2:scientific:learner3:end`

Positive lookbehind assertion can help us to find all words `'scientific'`, `'scientific'` and `'end'` preeceded by the words `learner{1-3}`.

```
import re

string = "begin:learner1:scientific:learner2:scientific:learner3:end"
print re.findall(r"(?<=learner\d:)(\b\w*\b)", string)
```

Neagative Lookbehind

Similar to positive lookbehind, `(?<!regex)` matches at a position if the pattern inside the lookbehind cannot be matched ending at that position.

inside the lookbehind cannot be matched ending at that position.

Example

Let's now proceed to an example, where we find the word (`begin`), not preceded by the words `learner{1-3}`.

```
import re

string = "begin:learner1:scientific:learner2:scientific:learner3:end"
print re.findall(r"^(?<!learner\d:)(\b\w*\b)", string)
```

Greedy/Non-Greedy Matching

Python Greedy and non-greedy matching explained with examples.

There are times when you want to match a pattern only optionally! The `?` character flags the group that precedes it as an optional part of the pattern. For example, enter the following into the interactive shell:

```
import re

Regex = re.compile(r'(scientific )?programming')
m1 = Regex.search('Learn programming')
m2 = Regex.search('Learn scientific programming')

print m1.group()
print m2.group()
```

The output will be:

```
programming
scientific programming
```

This means that the `(scientific )?` part of the regular expression means that the pattern `scientific` (notice the white space!) is an **optional** group. The regex will match text that has zero instances or one instance of `scientific` in it. This is why the regex matches both 'programming' and 'scientific programming'.

Note that the '`*`', '`+`', and '`?`' qualifiers are all **greedy**; they match as much text as possible. Sometimes this behavior isn't desired; if the RE pattern `<.*>` is matched against '`<H1>Learn Scientific Programming</H1>`', it will match the entire string, and not just '`<H1>`'. Adding '`?`' after the qualifier makes it perform the match in non-greedy or minimal fashion; as few characters as possible will be matched. Using `.*?` in the previous expression will match

only ‘`<H1>`’.

Project 1: Fun with DNA (REGEX Lookaround)!

In this project we find Opening reading frame or ORF from DNA sequences with the help of Python regex.

DNA is a sequence of bases, `A`, `C`, `G`, or `T`. They are translated into proteins 3-bases where each sequence is called a **codon**. There is a special start codon `ATG`, and three stop codons, `TGA`, `TAG`, and `TAA`. Example:

```
cgcgcATGcATGcgTGAcTAAcgTAGcgcgcgcgc
```

An opening reading frame or **ORF** consists of a **start codon**, followed by some more codons, and ending with a **stop codon**. The above example has overlapping ORFs.

- `ATGcATGcgTGA` and
- `ATGcgTGAcTAA`.

The following pattern only finds the first ORF (`atgcatgcgtga'`). Since it consumes the first ORF, it also consumes the beginning of the second ORF.

```
from re import *

dna = 'cgcgcATGcATGcgTGAcTAAcgTAGcgcgcgcgc'
dna = dna.lower()
orfpat = r'(?x) ( atg  (?: (?!tga|tag|taa) ... )*  (?:tga|tag|taa) )'
print findall(orfpat,dna)
```

We want to find an ORF without consuming it, we can use a **positive lookahead** assertion (`(?= ( atg`). We put the whole ORF pattern inside the lookahead and find the two `atgcatgcgtga` and `atgcgtgactaa`.

```
from re import *

dna = 'cgcgcATGcATGcgTGAcTAAcgTAGcgcgcgcgc'
dna = dna.lower()
```

```
dna = dna.lower()
orfpat = r'(?x) (?= ( atg  (?: (?!tga|tag|taa) ... )*  (?:tga|tag|taa) ))'
s = findall(orfpat,dna)

if s:
     print ', '.join(s)
```

This project **adopts** and **simplifies** the Splitsvile examples (DNA) from Rex Dwyer's ipython notebook.

Project 2: Parsing data from a HTML file with Python and REGEX

In this project we use REGEX to find some values from a HTML file

WE'LL COVER THE FOLLOWING

- Solution

In this project, we want to extract tabular information from a HTML file (see below). Our goal is to extract information available between and except the first numerical index (`1..6`).

Consider the `data.html` file below:

```
<html>
<head>
<style>
table, th, td {
    border: 1px solid black;
    border-collapse: collapse;
}
th, td {
    padding: 5px;
}
th {
    text-align: left;
}
</style>
</head>
<body>
<table style="width:100%">
<tr align="center"><td>1</td> <td>England</td> <td>English</td></tr>
<tr align="center"><td>2</td> <td>Japan</td> <td>Japanese</td></tr>
<tr align="center"><td>3</td> <td>China</td> <td>Chinese</td></tr>
<tr align="center"><td>4</td> <td>Middle-east</td> <td>Arabic</td></tr>
<tr align="center"><td>5</td> <td>India</td> <td>Hindi</td></tr>
```

```
<tr align="center"><td>6</td> <td>Thailand</td> <td>Thai</td></tr>
</table>

</body>
</html>
```

If we load the HTML file onto a browser it should look like below:

1	England	English
2	Japan	Japanese
3	China	Chinese
4	Middle-east	Arabic
5	India	Hindi
6	Thailand	Thai

A HTML file with table

Solution

In this code, we first extract HTML data (`data.html`) and then find and extract the values from the HTML code.

main.py

data.html

```
import re

with open('data.html', 'r') as myfile:
    data=myfile.read().replace('\n', '')

result=re.findall(r'<td>\w+</td>\s<td>(\w+)</td>\s<td>(\w+)</td>',data)
print(result)
```

Expected output :

```
[('England', 'English'),
('Japan', 'Japanese'),
('China', 'Chinese'),
('India', 'Hindi'),
('Thailand', 'Thai')]
```

Project 3: PDF scraping in Python + REGEX

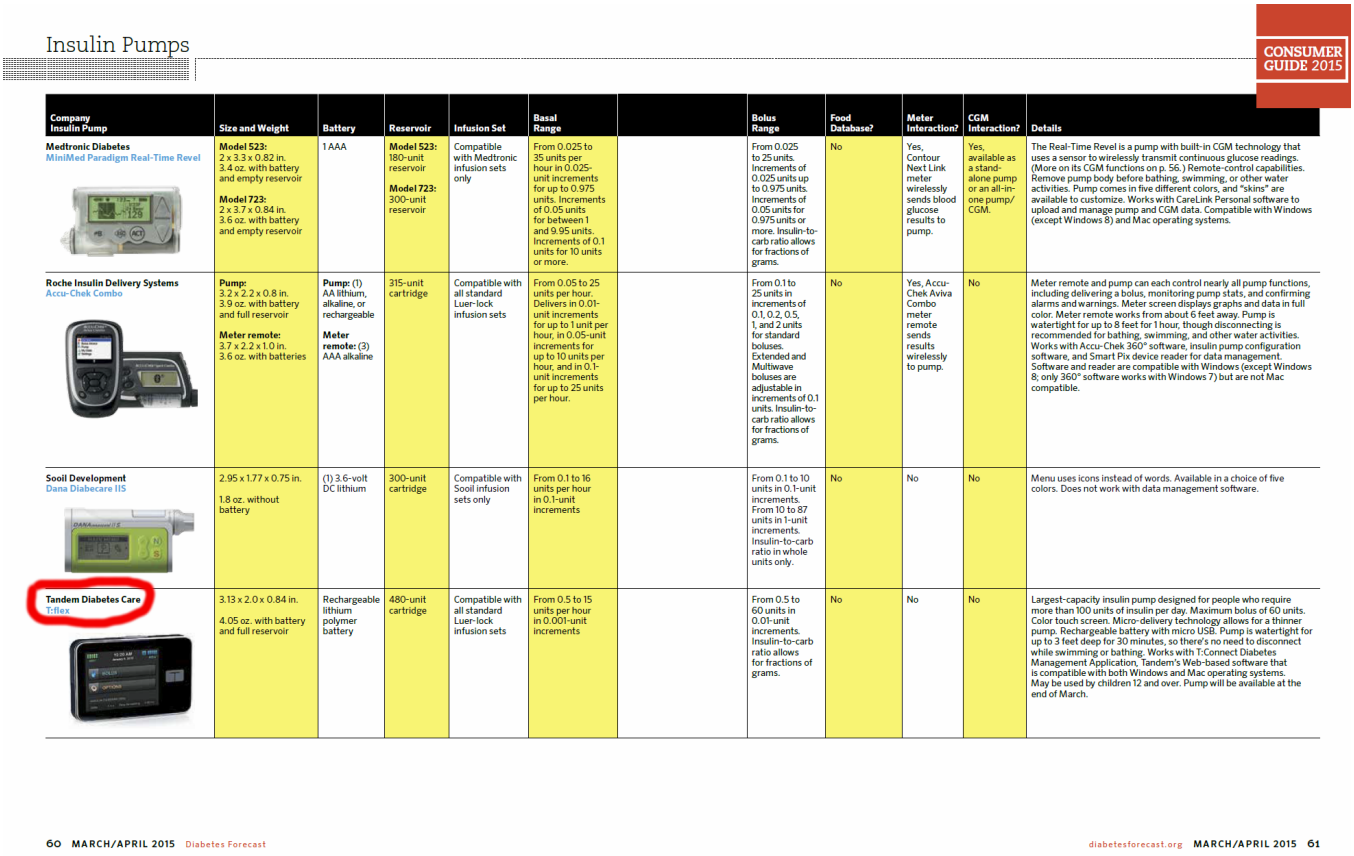
In this project we use regex to extract a list of items from a pdf file.

WE'LL COVER THE FOLLOWING

- PDF scraping example:
- Input file
- Solution

PDF scraping example:

In this project we will use a pdf file (see the screenshot below) from the diabetes.org website. Our goal is to list all the equipment models developed by the manufacturers names containing the word `tandem` (case insensitive).



Insulin Pumps

CONSUMER GUIDE 2015

Company Insulin Pump	Size and Weight	Battery	Reservoir	Infusion Set	Basal Range		Bolus Range	Food Database?	Meter Interaction?	CGM Interaction?	Details
Medtronic Diabetes MiniMed Paradigm Real-Time Revel	**Model 523:** 2 x 3.3 x 0.82 in. 3.4 oz. with battery and empty reservoir **Model 723:** 2 x 3.7 x 0.84 in. 3.6 oz. with battery and empty reservoir	1 AAA	**Model 523:** 180-unit reservoir **Model 723:** 300-unit reservoir	Compatible with Medtronic infusion sets only	From 0.025 to 35 units per hour in 0.025-unit increments for up to 0.975 units. Increments of 0.05 units for between 1 and 9.95 units. Increments of 0.1 units for 10 units or more.		From 0.025 to 25 units. Increments of 0.025 units up to 0.975 units. Increments of 0.05 units for 0.975 units or more. Insulin-to-carb ratio allows for fractions of grams.	No	Yes, Contour Next Link meter wirelessly sends blood glucose results to pump.	Yes, available as a stand-alone pump or an all-in-one pump/CGM.	The Real-Time Revel is a pump with built-in CGM technology that uses a sensor to wirelessly transmit continuous glucose readings. (More on its CGM functions on p. 56.) Remote-control capabilities. Remove pump body before bathing, swimming, or other water activities. Pump comes in five different colors, and "skins" are available to customize. Works with CareLink Personal software to upload and manage pump and CGM data. Compatible with Windows (except Windows 8) and Mac operating systems.
Roche Insulin Delivery Systems Accu-Chek Combo	**Pump:** 3.2 x 2.2 x 0.8 in. 3.9 oz. with battery and full reservoir **Meter remote:** 3.7 x 2.2 x 1.0 in. 3.6 oz. with batteries	**Pump:** (1) AA lithium, alkaline, or rechargeable **Meter remote:** (3) AAA alkaline	315-unit cartridge	Compatible with all standard Luer-lock infusion sets	From 0.05 to 25 units per hour. Delivers in 0.01-unit increments for up to 1 unit per hour, in 0.05-unit increments for up to 10 units per hour, and in 0.1-unit increments for up to 25 units per hour.		From 0.1 to 25 units in increments of 0.1, 0.2, 0.5, 1, and 2 units for standard boluses. Extended and Multiwave boluses are adjustable in increments of 0.1 units. Insulin-to-carb ratio allows for fractions of grams.	No	Yes, Accu-Chek Aviva Combo meter remote sends results wirelessly to pump.	No	Meter remote and pump can each control nearly all pump functions, including delivering a bolus, monitoring pump stats, and confirming alarms and warnings. Meter screen displays graphs and data in full color. Meter remote works from about 6 feet away. Pump is watertight for up to 8 feet for 1 hour, though disconnecting is recommended for bathing, swimming, and other water activities. Works with Accu-Chek 360° software, insulin pump configuration software, and Smart Pix device reader for data management. Software and reader are compatible with Windows (except Windows 8; only 360° software works with Windows 7) but are not Mac compatible.
Sooil Development Dana Diabecare IIS	2.95 x 1.77 x 0.75 in. 1.8 oz. without battery	(1) 3.6-volt DC lithium	300-unit cartridge	Compatible with Sooil infusion sets only	From 0.1 to 16 units per hour in 0.1-unit increments		From 0.1 to 10 units in 0.1-unit increments. From 10 to 87 units in 1-unit increments. Insulin-to-carb ratio in whole units only.	No	No	No	Menu uses icons instead of words. Available in a choice of five colors. Does not work with data management software.
Tandem Diabetes Care T:flex	3.13 x 2.0 x 0.84 in. 4.05 oz. with battery and full reservoir	Rechargeable lithium polymer battery	480-unit cartridge	Compatible with all standard Luer-lock infusion sets	From 0.5 to 15 units per hour in 0.001-unit increments		From 0.5 to 60 units in 0.01-unit increments. Insulin-to-carb ratio allows for fractions of grams.	No	No	No	Largest-capacity insulin pump designed for people who require more than 100 units of insulin per day. Maximum bolus of 60 units. Color touch screen. Micro-delivery technology allows for a thinner pump. Rechargeable battery with micro USB. Pump is watertight for up to 3 feet deep for 30 minutes, so there's no need to disconnect while swimming or bathing. Works with T:Connect Diabetes Management Application, Tandem's Web-based software that is compatible with both Windows and Mac operating systems. May be used by children 12 and over. Pump will be available at the end of March.

60 MARCH/APRIL 2015 Diabetes Forecast

diabetesforecast.org MARCH/APRIL 2015 61

Find all the product models by the manufacturer called `Tandem`

Input file

You can download the input file from here: data.pdf.

Solution

A complete explanation of the Python code is **out of the scope** for this course (hint: learn the Python module `pdfquery`). It should be easy enough for you to understand how we capture the `porduct_name` from the pdf file using bounding box function `LTTextLineHorizontal:in_bbox("40, 48, 181, 633")` and then iterate over the products and search using regex and then only print the `Tandem` Manufacurers.

```
import re

import pdfquery
from lxml import etree


PDF_FILE = 'data.pdf'

pdf = pdfquery.PDFQuery(PDF_FILE)
pdf.load()

product_info = []
page_count = len(pdf._pages)
for pg in range(page_count):
    data = pdf.extract([
        ('with_parent', 'LTPage[pageid="{}"]'.format(pg+1)),
        ('with_formatter', None),
        ('product_name', 'LTTextLineHorizontal:in_bbox("40, 48, 181, 633")'),
    ])

    for ix, pn in enumerate(sorted([d for d in data['product_name'] if d.text.strip()], key=l
        if ix % 2 == 0:
            product_info.append({'Manufacturer': pn.text.strip(), 'page': pg, 'y_start': floa
            if ix > 0:
                product_info[-2]['y_end'] = float(pn.get('y0'))+10.0
        else:
            product_info[-1]['Model'] = pn.text.strip()

pdf.file.close()


for p in product_info:
    s = p['Manufacturer']
    m = re.search(r"Tandem",s,re.I)
    if m:
        print('Manufacturer: {}[Model {}]\n'.format(p['Manufacturer'],p['Model']))
```

We have preloaded the data onto educative.io's server and you should be able to run the code straight ahead and get the output as follows:

```
Manufacturer: Tandem Diabetes Care[Model T:flex]
Manufacturer: Tandem Diabetes Care[Model T:slim]
```

From this result we can see that there are two models `T:fles` and `T:slim` supplied by the manufacturer called 'Tandem Diabetes Care'. The problem solution has been adopted and simplified from the reddit user insainodwayno.

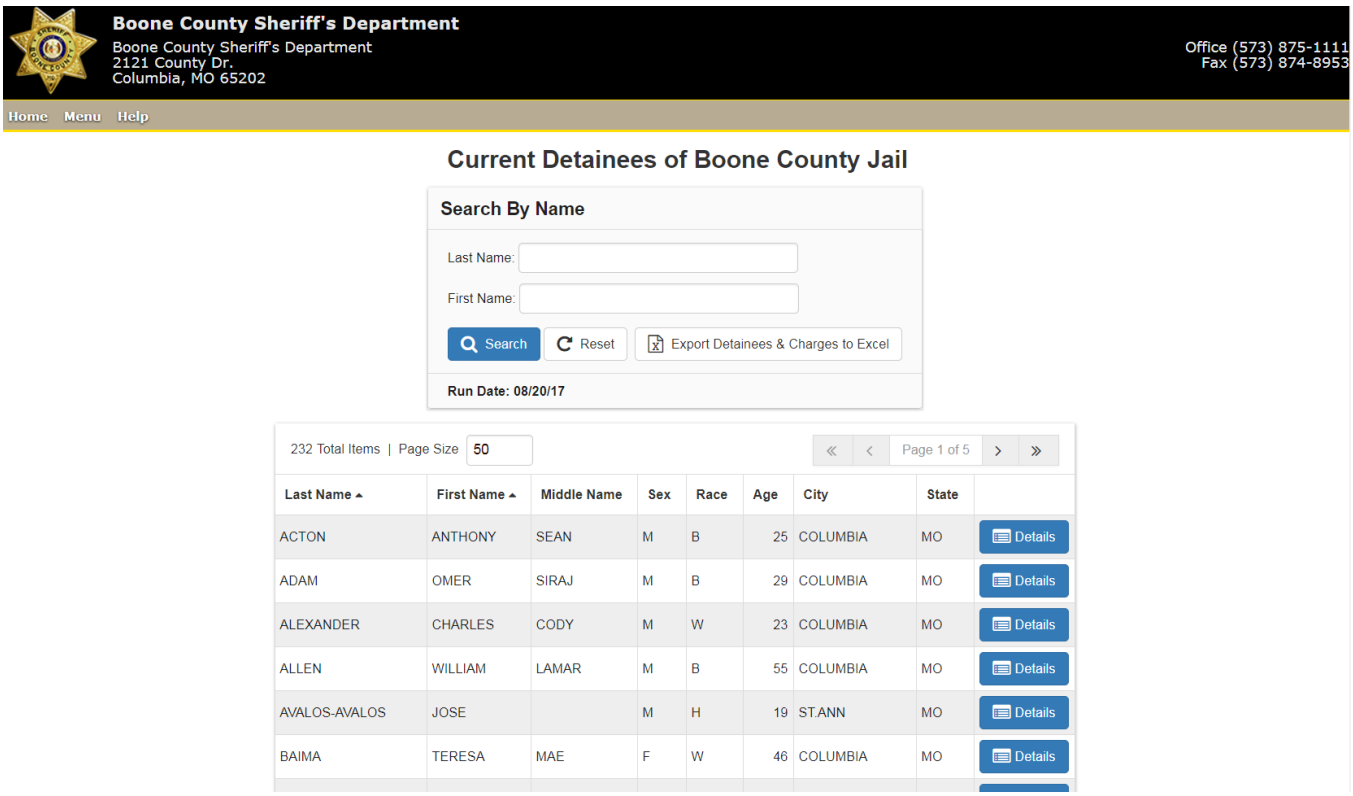
Project 4: Web scraping in Python + REGEX

In this project we use BS- BeautifulSoup and REGEX to find some whose last name starts with the letter 'A'

WE'LL COVER THE FOLLOWING

- Solution

Web scraping or web data extraction is data scraping used for extracting data from websites. In this project, we will extract tabular data from the Boone Country Sherrif's Dept website of criminal records (see the image below) and then find all the people whose name start with the word "A".



Solution

The problem solution uses BeautifulSoup. A detailed explanation of the code is out-of-scope for this course (hint: read the BS docs). In this code, we first extract HTML data and format/convert into BS's table using BS's `BeautifulSoup()` function, then find and extract the table from the HTML

code.

```
import re
from pprint import pprint
import csv
import requests
from bs4 import BeautifulSoup


url = 'http://www.showmeboone.com/sheriff/JailResidents/JailResidents.asp'
response = requests.get(url)
html = response.content

soup = BeautifulSoup(html)
table = soup.find('tbody', attrs={'class': 'stripe'})

list_of_rows = []

for row in table.findAll('tr')[0:]:
    list_of_cells = []
    for cell in row.findAll('td'):
        text = cell.text.replace(' ', '')
        list_of_cells.append(text)
    list_of_rows.append(list_of_cells)

for line in list_of_rows:
    row = '\t'.join(str(i) for i in line)  # python 2
    s=row[0:5] # Select only the Last names (1st column)
    m = re.search(r"^A",s,re.I)
    if m:
      print row
```

Expected output (Surnames started with the letter "**A**"):

```
ACTON    ANTHONY SEAN    M   B   25  COLUMBIA    MO  Details
ADAM     OMER    SIRAJ   M   B   29  COLUMBIA    MO  Details
ALEXANDER   CHARLES CODY    M   W   23  COLUMBIA    MO  Details
ALLEN    WILLIAM LAMAR   M   B   55  COLUMBIA    MO  Details
AVALOS-AVALOS   JOSE        M   H   19  ST.ANN  MO  Details
```

Easy!

This examples has been **adopted and extended** from the Python-BeautifulSoup's first web scraper, originally developed by Chase Davis, Jackie Kazil, Sisi Wei and Matt Wynn for bootcamps held by Investigative Reporters and Editors at the University of Missouri in Columbia, Missouri.

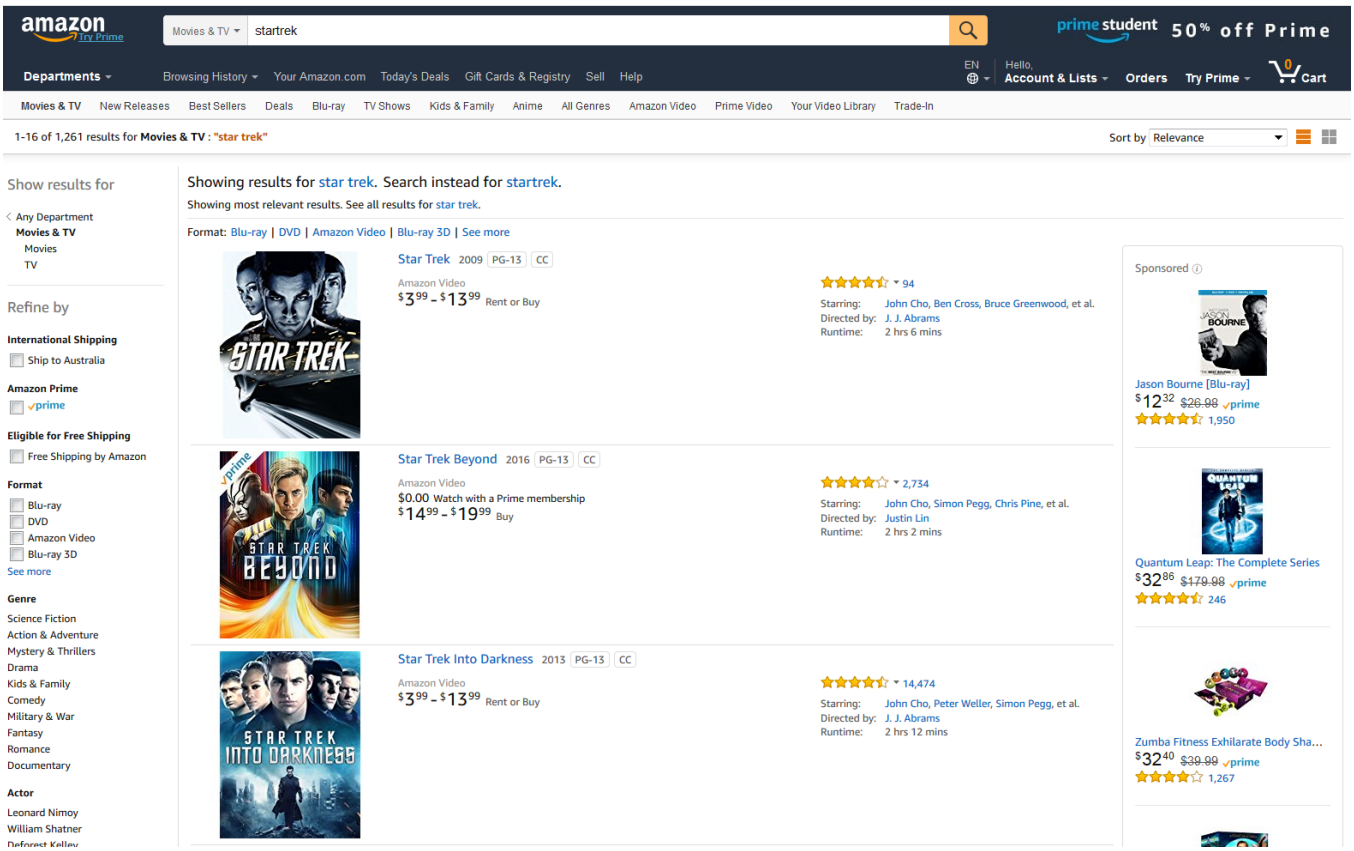
Project 5: Amazon web crawling in Python + REGEX

In this project we use BS- BeautifulSoup and REGEX to find some products by crawling an Amazon.com page

WE'LL COVER THE FOLLOWING

- Solution

A **Web crawler**, sometimes called a spider, is an Internet bot that systematically browses the World Wide Web, typically for the purpose of Web indexing (web spidering). In this project, we crawl the amazon.com website > Movies & TV > 'startrek' (see the image below). Then, we find the list of movies with 'bonus' content.



Amazon web scraping for Startrek DVD movies with `bonus' content

Solution

The problem solution uses BeautifulSoup. A detailed explanation of the code is

out-of-scope for this course (hint: read the BS docs). In this code, we first

extract HTML data and format/convert into BS's table using BS's `BeautifulSoup()` function, then find and extract the movies from the HTML code.

```
import re
from pprint import pprint
import csv
import requests


import requests
from bs4 import BeautifulSoup
def crawl_amazon_web(page,WebUrl):
    if(page>0):
        url = WebUrl
        code = requests.get(url)
        plain = code.text
        s = BeautifulSoup(plain, "html.parser")

        for link in s.findAll('a', {'class':'s-access-detail-page'}):
            movie_title = link.get('title')
            m = re.search( r'Bonus',movie_title)
            if m:
              print(movie_title)
              html_link = link.get('href')
              print(html_link)

crawl_amazon_web(1,'https://www.amazon.com/s/ref=nb_sb_noss_2?url=search-alias%3Dmovies-tv&fi
```

Expected output (Startrek movies with bonus content) :

```
Star Wars: The Force Awakens (Plus Bonus Features)
https://www.amazon.com/Star-Wars-Force-Awakens-Features/dp/B019EG1TC8
Rogue One: A Star Wars Story (With Bonus Content)
https://www.amazon.com/Rogue-One-Story-Bonus-Content/dp/B01N7FYJ7H
```

Easy!

This solution has been **adopted and extended** from the Dev.to post written by Pranay Das.

Python materials

- **Python REGEX**
  - Python Regular expression operations - Official doco.
  - Regular Expression HOWTO by A.M. Kuchling
- **Educative courses**
  - Python 3: An interactive deep dive by Mark Pilgrim, implemented by the Educaitve Team
  - Python 101: Interactively learn how to program with Python 3 by Michael Driscoll
  - Python 201 - Interactively Learn Advanced Concepts in Python 3 by Michael Driscoll
- **Online Python Courses**
  - edx's Introduction to Computer Science and Programming Using Python . The companion book can be found here.
    - MIT Open Courseware also offers a gentler "lead-in" course designed for those with no programming background that you can take beforetaking the above: Building Programming Experience: A Lead-In to 6.001.
  - MIT Open Courseware's A Gentle Introduction to Programming sing Python
  - Coursera's Programming for Everybody (Python) For beginners.
  - Codecademy's beginners; tends to focus primarily on syntax Python track.
  - Udacity's Programming Foundations with Python. Focuses on object-

    - Udacity's Programming Foundations with Python. Focuses on object oriented programming.
    - Team Treehouse's Python course.
    - Udemy's Python Regular Expressions with Data Scraping Projects, this book - video version!
- **Interactive resources:**
    - LearnPython An interactive online guide that teaches basic Python.
    - Try Python Another interactive online guide.
    - Educative's Python Regular Expressions with Data Scraping Projects, this course!
- **Books and tutorials (online):**
    - Learn Python the Hard Way Part of the "Learn X the Hard Way" series. Despite its name, this is one of the easiest introductions to Python available.
    - Automate the Boring Stuff with Python .From the Invent with Python author.
    - How to Think Like a Computer Scientist (Python 2 version and Python 3 version)
- Think PythonComprehensive introductory text on Python.

```
-   The official Python tutorial (for [Python 2](https://docs.python.org/2
/tutorial/) and [Python 3](https://docs.python.org/3/tutorial/)). Moves
 a little quickly, but is very comprehensive and thorough.

-   [Problem Solving with Algorithms and Data Structures](http://interacti
vepython.org/runestone/static/pythonds/index.html)

-   [Dive into Python 3](http://www.diveintopython3.net/) An accelerated i
ntroduction to Python.

-   [Program Arcade Games With Python And Pygame](http://ProgramArcadeGame
s.com)

-   [The Hitchhiker's Guide to Python](https://python-guide.readthedocs.or
```

```
g/en/latest/)


-   [pycrumbs](http://resrc.io/list/4/pycrumbs/) A huge list of many usefu
l articles, tutorials, and snippits on Python, ranging from basic t
o        advanced.

-   [PyMOTW](http://pymotw.com/) A tour of the Python standard library
    through short examples.
-   [Import Python](http://importpython.com/books/) A catalog of Python bo
oks (some are free)
```

- **Exercises:**
    - Pyschools Exercises and challenges in Python.